Thapar Institute of Engineering and Technology

Patiala

UCS503P — Course Project

Janmitra

A Voice-First Civic Scheme Guidance Platform for Rural India

Prototype-Stage Project Report

Mid-Semester Evaluation

Submitted to

Dr Paramveer Kaur

Submitted by

Student	**Roll number**
Dhruv Srivastava	1024170394
Harkamal Singh Lubana	1024170396
Paras	1024170395

dsrivastava_be24@thapar.edu

hlubana_be24@thapar.edu

pparas_be24@thapar.edu

Computer Science and Engineering Department

10 October 2026

Abstract

Janmitra is a voice-first civic-guidance project intended to help rural citizens discover government support, understand eligibility conditions, prepare documents, and request human assistance. This report describes the implemented prototype at repository revision `e34544d`. The current interaction channel is browser audio over LiveKit, with Gemini Live handling native speech input and output. A FastAPI backend provides structured scheme retrieval, deterministic eligibility evaluation, document checklists, versioned publication, an operator handoff queue, and conversation and audit records.

The prototype includes 18 structured scheme records across six categories. All 18 have been human-verified, as confirmed by the project team; five contain rule sets. The bundled JSON review metadata still requires synchronisation with this completed review, while verified eligibility and checklist tools require published records. A local backend verification run completed with 310 passing tests in the selected suite; two test modules requiring optional voice-runtime dependencies were excluded. Lint checks passed. Repository documentation also records one successful synthetic English voice call, alongside intermittent failures and unresolved answer-quality gaps.

The work establishes a testable application foundation and a browser voice prototype. Phone/SIP access from the original proposal, complete source ingestion, publication of the reviewed catalogue, representative multilingual evaluation, measured load scaling, and staging deployment remain outstanding. This report distinguishes implemented mechanisms, observed results, and future objectives.

Contents

Abstract **i**

1 Introduction **1**

1.1 Background and Context 1

1.1.1 Problem Statement 1

1.2 Project Objectives 2

1.2.1 Time-to-Value and Development Approach 2

1.2.2 Prototype Scope and Changes from the Proposal 2

2 Methodology and Design **4**

2.1 System Architecture 4

2.1.1 High-Level Design 4

2.1.2 Technical Stack 5

2.2 Implementation Details 5

2.2.1 Voice Conversation and Session Lifecycle 5

2.2.2 Typed Backend Tools and Request Flow 5

2.2.3 Canonical Scheme Records and the Reference Catalogue 6

2.2.4 Retrieval and Grounding 6

2.2.5 Deterministic Eligibility and Document Checklists 7

2.2.6 Versioned Publication and Review Workflow 7

2.2.7 Human Handoff and Operator Queue 8

2.2.8 Persistence, Audit, and Access Control 8

2.2.9 Development, CI, and Deployment Foundations 8

3 Results and Evaluation **9**

3.1 Testing Methodology 9

3.1.1 Unit Testing 9

3.1.2 Integration Testing 9

3.1.3 Local Verification for This Report 9

3.1.4 Voice, Performance, and User Testing 10

3.2 Performance Metrics 10

3.2.1 Definition and Interpretation of Timing 10
3.2.2 Observed Results and Evaluation Targets 11
3.3 Analysis 12
3.3.1 Achievement of Prototype Objectives 12
3.3.2 Recent Development Progress 12
3.3.3 Challenges and Design Responses 12
3.3.4 Limitations at the Prototype Checkpoint 13

4 Conclusion **14**
4.1 Summary of Work 14
4.2 Key Findings 14
4.3 Future Work and Recommendations 15
4.4 Final Remarks 15

Bibliography **16**

Chapter 1

Introduction

1.1 Background and Context

Government welfare information is distributed across departmental websites, guidelines, notices, and application portals. A citizen may describe a practical need—such as support for farming, employment, housing, insurance, or a small business—without knowing the relevant scheme name or department. Formal language, unfamiliar terminology, and multi-page navigation can make the information difficult to use even when it is publicly available.

The higher-order goal of Janmitra is to improve access to this information through conversation. The original proposal envisaged a service reachable by an ordinary phone call, allowing citizens to speak in a familiar language or dialect without installing an application [1]. The prototype develops the speech and backend workflow through a browser voice channel. This provides a practical environment for integration and demonstration, while retaining phone accessibility as an unmet objective of the original proposal.

1.1.1 Problem Statement

The project addresses five connected difficulties:

1. **Fragmented information:** relevant facts may be spread across multiple official sources and versions.
2. **Language and literacy barriers:** a citizen may be more comfortable describing a need aloud than reading formal scheme descriptions.
3. **Discovery difficulty:** a person may know their circumstances but not the category or name of a suitable programme.
4. **Eligibility and document uncertainty:** general descriptions do not always explain which conditions apply to a particular person or what they should prepare before visiting an office.
5. **Incomplete assistance:** a self-service interaction needs an explicit route to human help when the issue cannot be resolved.

The engineering problem is to combine a natural spoken interaction with traceable information and predictable backend decisions. Language generation alone cannot establish personal eligibility, official approval, or completion of an application. These distinctions guide the prototype's tool boundaries.

1.2 Project Objectives

The prototype checkpoint is organised around the following observable objectives. Their implementation and validation status are examined in Chapter 3.

1. **Demonstrate the voice-to-tool path:** connect browser audio to LiveKit and Gemini Live, retrieve scheme context through the API, and return a spoken response during a controlled demonstration.
2. **Define a reusable catalogue:** represent scheme descriptions, citations, review state, documents, application steps, and optional rules using one validated schema. Include an initial, extensible reference set.
3. **Implement deterministic guidance:** evaluate rule conditions and conditional document requirements with explicit outcomes and boundary tests.
4. **Preserve accountable publication:** require verified records for publication, retain previous versions, and associate verification with a named reviewer and date.
5. **Support human assistance:** create a queued handoff through a validated backend operation and expose operator status updates and context.
6. **Establish repeatable verification:** provide unit and API tests, linting, database migrations, container definitions, and voice-evaluation tools before expanding pilot and performance testing.

1.2.1 Time-to-Value and Development Approach

The proposal prioritised an early working voice loop, followed by catalogue expansion, ingestion, and load testing. The prototype follows an incremental approach: establish data contracts and domain services, integrate voice through HTTP tools, add representative source records, then strengthen retrieval, concurrency controls, and evaluation support. Each increment provides a concrete artefact that can be inspected or tested rather than relying on a final integrated demonstration alone.

1.2.2 Prototype Scope and Changes from the Proposal

The report covers the current repository implementation [2], associated technical documentation, and the local checks described in Chapter 3. It does not assume that an externally hosted deployment or a populated production database exists.

The browser channel currently requires a compatible device and data connection. It therefore does not yet deliver the proposal's access from an ordinary phone. All 18 initial scheme records have been human-verified, but completed human review must be distinguished from recorded verification metadata and database publication. These scope differences are material to evaluating the prototype fairly [3, 4].

Area	Original proposal	Current prototype
Citizen access	Ordinary telephone call through a SIP bridge.	Browser microphone and speaker through LiveKit; no telephone integration.
Scheme guidance	Facts obtained from a verified, published catalogue.	All 18 initial records are human-verified; review metadata and publication remain to be synchronised. Qualified general guidance is allowed when context is incomplete.
Eligibility	Deterministic pre-checks over reviewed rules.	Engine and tests implemented; published records with rule sets are required for deterministic checks.
Source ingestion	Import, AI extraction, human review, and versioned publish.	Structured JSON preparation, review attestation, and versioned publication; complete ingestion workflow remains future work.
Delivery and scale	Staging pipeline and measured replica comparisons.	CI checks, containers, migrations, and concurrency tests; staging CD and load measurements remain outstanding.

Table 1.1: Relationship between the proposal and the prototype checkpoint.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

Janmitra uses a modular backend application with a separate voice worker. The backend contains conversation, catalogue, eligibility, handoff, and audit modules. The voice worker calls the backend through typed HTTP endpoints and does not duplicate eligibility rules or write directly to the database.

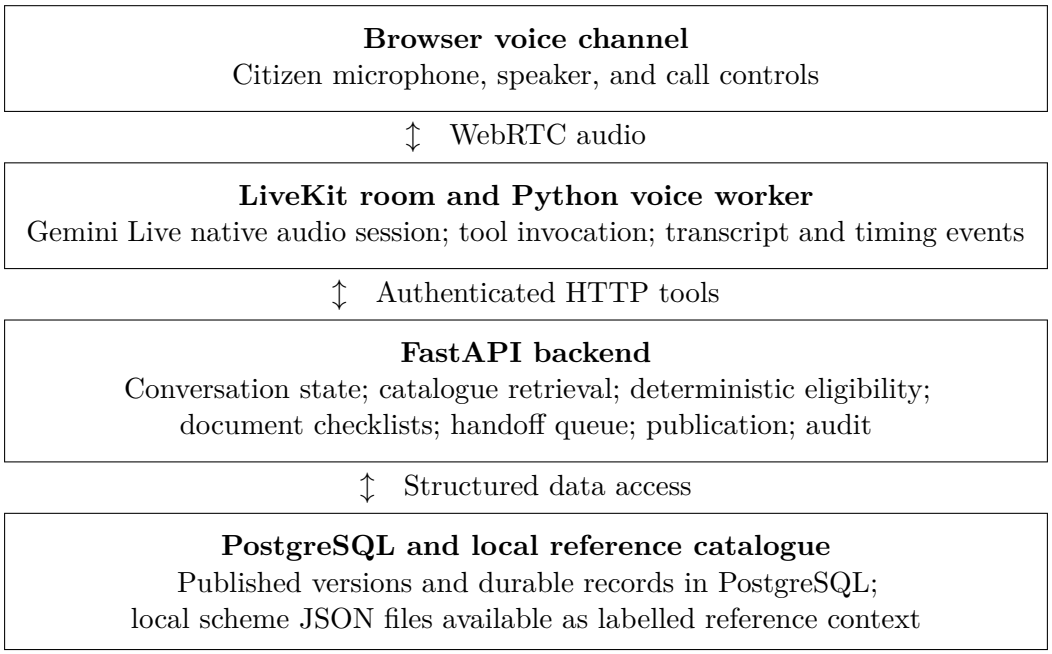


Figure 2.1: Implemented prototype architecture. Admin and operator API clients access the backend separately. A phone/SIP bridge is not part of this implementation.

The database preserves application records across API requests. The active real-time audio session remains the responsibility of its voice worker. This separation supports independent growth in API and voice-worker capacity, but a measured scaling claim requires the load experiments described later.

2.1.2 Technical Stack

- **Backend language and framework:** Python 3.12 or later, FastAPI, and Uvicorn for asynchronous HTTP services.
- **Validation and persistence:** Pydantic request and record models, SQLAlchemy asynchronous sessions, PostgreSQL, the asyncpg driver, and Alembic migrations. SQLite supports isolated local tests.
- **Voice integration:** LiveKit Agents and its Google plugin are pinned to version 1.8.5. The configured speech model is `gemini-3.1-flash-live-preview`.
- **Model separation:** Gemini Live handles the spoken exchange; a separate text adapter supports backend text tasks. Mock and failure adapters support controlled tests without real-provider calls.
- **Integration and delivery tools:** HTTPX, pytest, pytest-asyncio, Ruff, Docker Compose, and GitHub Actions.

These are the technologies declared by the checked revision. Dependency configuration is evidence of the intended runtime, not evidence that every component has been deployed or benchmarked.

2.2 Implementation Details

2.2.1 Voice Conversation and Session Lifecycle

The worker connects to a LiveKit room, creates a backend conversation, and starts an `AgentSession` with Gemini Live. Native audio input and output are handled within the same model session; the implementation does not introduce a separate speech-to-text service followed by a separate text-to-speech service.

The current flow is citizen-initiated: the citizen speaks first, and the prompt requests a short AI disclosure in the initial spoken reply. The agent is instructed to use the citizen's latest language, keep spoken steps understandable, and ask one question at a time. These are behavioural instructions whose quality must still be checked in real conversations.

The worker configures voice activity detection with 100 ms prefix padding and 500 ms silence endpointing. The default thinking level is `low`, configurable within the same live model. These configuration values are not measured response latencies. Transcripts and timing events are written asynchronously, and cleanup drains pending writes and closes the backend conversation through a single cleanup task. Safe status messages communicate readiness and recovery state to the client.

2.2.2 Typed Backend Tools and Request Flow

The agent exposes four primary operations:

`find_service`
: Retrieves relevant scheme context for the citizen's stated need, optionally narrowed by category.

`check_eligibility`
: Evaluates submitted answers against the selected published record's deterministic rule set.

`get_documents`
: Produces the published record's checklist, including conditional requirements.

`request_handoff`
: Requests human assistance using contact details, an issue summary, and reported escalation signals.

Each tool request identifies the conversation and requested language. The backend validates the payload, checks the caller's role, retrieves the conversation, and performs the appropriate domain operation. Scheme facts, eligibility results, and handoff records are returned as structured responses. The model explains those results in speech; it cannot directly write a database publication or substitute its own verdict for the eligibility engine's result.

2.2.3 Canonical Scheme Records and the Reference Catalogue

A `ServiceRecord` contains a stable slug, localised name and description, aliases, category, benefit and eligibility summaries, optional rule set, documents, application steps, and citation metadata. The citation identifies the source, publisher, review state, verification date, and reviewer when verified.

The current reference directory contains 18 JSON records. Their category distribution is shown in Table 2.1. All 18 have been human-verified, as confirmed by the project team. The bundled files still carry `pending_review` metadata and no named reviewer; this stored metadata must be synchronised with the completed review before verified publication. Five records—APY, PM-SYM, PMJJBY, PMSBY, and PMUY—contain rule sets; 15 records contain nonempty document lists. The remaining three do not yet supply document lists. Human verification does not by itself establish database publication in a deployment.

Category	Records	Record identifiers
Loan	4	KCC, PMMY, PMEGP, PM-Vishwakarma
Banking	2	PMJDY, DAY-NRLM
Grant	4	PM-KISAN, PMAY-G, PMGKAY, PMUY
Insurance	4	PM-JAY, PMFBY, PMJJBY, PMSBY
Pension	3	APY, PM-SYM, NSAP
Other	1	Rural-employment reference with stable slug `mgnrega`
Total	**18**	All human-verified

Table 2.1: Inventory of the 18 human-verified scheme records. Review completion is confirmed by the project team; repository metadata still reflects the earlier state.

The catalogue review checklist requires comparison of amounts, ages, exclusions, documents, and application steps against official sources. Automated source-check notes assist that work but explicitly do not constitute human approval [6, 7]. The project team confirms that human verification of all 18 initial scheme records is complete. Reviewer attribution and recorded review status remain to be synchronised in the catalogue files.

2.2.4 Retrieval and Grounding

The recent retrieval implementation combines Unicode-aware normalisation, multilingual phrase expansion, named-scheme matching, and a curated mapping from expressed needs to candidate schemes. It attempts to preserve distinct needs in a compound request rather than returning

several variants of the same topic. This is lightweight retrieval over structured records; it does not use an embedding service or a vector database.

The voice client requests `response_mode="context"`. The API combines published records with local references, replaces a local record when a published version of the same slug exists, and returns at most three selected records in total. The `context_order` field preserves ordering across published `matches` and `reference_context`. Citations, service versions where available, and stored verification status accompany the context. Until the review metadata is synchronised, the API still labels local records as pending despite the completed human review.

Gemini Live then generates the spoken answer directly from this result, avoiding an additional text-model answer-generation round trip in the voice path. A `prepared_answer` mode remains for compatible HTTP clients. The current prompt permits qualified general orientation from trained knowledge when retrieved context is incomplete, while forbidding invented current amounts, deadlines, confirmed eligibility, and completed actions. This broadens the original proposal's verified-record-only guidance policy and requires explicit quality evaluation [4].

2.2.5 Deterministic Eligibility and Document Checklists

Eligibility rules are stored as data within a versioned scheme record. Questions declare answer types and optional limits. Named conditions contain comparisons, and optional `applies_when` guards determine when a condition is relevant. The decision combines conditions through `all_of`, `any_of`, and `none_of` groups.

The engine validates answers before evaluation and returns one of three states: `eligible`, `not_eligible`, or `needs_more_info`. A missing answer remains unknown, allowing the backend to supply the next relevant question. Condition traces identify the tests involved in the result and may include supporting source text. Numeric validation rejects inappropriate types, non-finite values, and values outside declared bounds.

For illustration only, a synthetic rule might require an applicant to be at least 18 years old. An answer below that boundary fails the condition; an omitted age requests more information. This example explains engine behaviour and is not a claim about a real scheme.

Document checklists use the same structured answers. An unconditional document is included directly. A document whose condition is false is omitted; when the condition is unknown, it remains visible as conditional. The verified endpoints require a published record, and eligibility also requires an associated rule set. Human-reviewed rules must also be published before these endpoints can use them.

2.2.6 Versioned Publication and Review Workflow

The `Service` table holds scheme identity and a pointer to its current version. `ServiceVersion` holds the payload, version number, publication state, author, notes, and timestamp. Publishing appends a new version and updates the pointer while retaining earlier versions. History and field-level diffs are available through admin routes.

The seed command validates local JSON records. To publish pending records, the reviewer supplies a name using `-actor` and explicitly attests to review using `-confirm-reviewed`. The command then marks the records verified for publication. The reviewer must actually check the sources before giving this attestation; the flag itself cannot prove factual correctness. PostgreSQL advisory and row locks serialize publication for a scheme, including its first publication.

The current workflow is local record preparation, human review, and CLI/API publication. A complete official-source importer, extraction-to-draft workflow, and integrated review dashboard

are not established by this backend. A source snapshot model and model-adapter drafting method provide foundations, but the full ingestion process remains future work.

2.2.7 Human Handoff and Operator Queue

The handoff service evaluates reported signals such as a citizen request for a person, an out-of-scope request, no matching scheme, low confidence, or repeated tool problems. A successful request stores an issue summary, optional contact name and phone number, category, language, and conversation reference. The conversation is marked `handed_off`.

Operators can list queued requests, inspect the related conversation events, add notes, and move a request through allowed states. The principal progression is `new` to `contacted` to `resolved`; direct resolution from `new` is also allowed. A queued handoff is a record for subsequent operator attention. The implementation does not transfer a live telephone call or guarantee that a callback has occurred.

2.2.8 Persistence, Audit, and Access Control

The principal database entities are services, service versions, source snapshots, conversations, conversation events, handoff requests, and audit events. Alembic provides an initial schema and a subsequent migration aligning structured payload columns with PostgreSQL JSONB. Category and identity lookups have relational indexes; current retrieval still ranks candidate records in application code, so JSONB storage alone does not establish indexed full-catalogue search capacity.

Conversation events retain ordered transcript and tool context. Audit events record critical actions with request identifiers, actors, and entity references. Conversation row locks serialize related state transitions and event allocation in PostgreSQL. Admin, operator, and voice API keys restrict access by role. This is a prototype access-control mechanism rather than a complete user-account and organisational permissions system.

The prompt prohibits requesting Aadhaar numbers, full bank details, OTPs, PINs, and passwords. Contact capture is optional and associated with handoff. However, transcripts are persisted, and prompt instructions alone cannot guarantee that unsolicited sensitive information is never stored. Redaction, consent, retention, and operator-access procedures require further work before a citizen pilot.

2.2.9 Development, CI, and Deployment Foundations

The backend GitHub Actions workflow installs development, voice, and Gemini dependencies, runs Ruff and pytest, and provisions PostgreSQL 16 for integration tests. These tests cover migrations and concurrency behaviour beyond what SQLite can establish. Workflow configuration documents the intended checks; the local results reported here do not substitute for a retrieved CI run result.

Docker Compose defines PostgreSQL and the API, with health checks and a persistent database volume. The API container applies migrations before starting Uvicorn. The voice worker is started separately. The repository provides deployment foundations, but no completed staging CD pipeline or measured multi-replica deployment is claimed in this report.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

3.1.1 Unit Testing

The tests exercise rule comparisons and decision combinations, missing answers, numeric validation, conditional documents, catalogue ranking, schema constraints, review attestation, and handoff trigger selection. Mock adapters make expected outputs reproducible and avoid provider cost. Synthetic test records are clearly separate from the government-scheme catalogue; their conditions are test fixtures and must not be reused as verified public guidance.

3.1.2 Integration Testing

HTTP tests run the FastAPI application against isolated SQLite databases. They exercise role restrictions, published-record retrieval, reference context, eligibility and document responses, handoff creation and updates, conversation events, and audit data. PostgreSQL-specific tests additionally cover migration round trips, model/schema agreement, concurrent publications, event numbering, write-once guidance timestamps, and competing terminal state transitions.

The prototype also contains voice-client, agent, and worker tests. Worker tests use controlled runtime substitutes to exercise cleanup, notifications, and session behaviour. These tests help validate orchestration but cannot demonstrate real speech quality or provider reliability.

3.1.3 Local Verification for This Report

A local Python 3.12 run on 10 October 2026 checked the current revision with model calls mocked and configuration-file loading disabled. It produced **310 passing tests**, with no failed tests in that selected run. PostgreSQL-specific checks were outside the executed selection because they require an explicitly configured PostgreSQL test database.

The two modules `tests/test_voice_agent.py` and `tests/test_voice_worker.py` were excluded because the temporary verification environment did not contain the optional LiveKit/Google runtime dependencies. Other voice-related checks, including prompt, client, and model-setting tests, were included. Thus, 310 passing tests describes the executed selection, not a claim that every repository test or a live voice deployment passed.

Ruff passed over `app`, `tests`, `scripts`, and `alembic`. Two non-failing warnings were emitted by the test run: a cache-setting warning caused by disabling pytest's cache plugin, and a dependency deprecation warning for an HTTP status constant. No credentials, real-provider

calls, or deployed database contents were needed for these checks.

3.1.4 Voice, Performance, and User Testing

Two scripts support separate evaluation needs:

- `scripts/smoke_voice.py` exercises the LiveKit voice flow and checks audio, retrieval, response, and conversation lifecycle behaviour.
- `scripts/evaluate_voice_answer.py` supports isolated native-audio answer evaluation using supplied WAV files and local retrieval. It uses real Gemini quota but does not exercise LiveKit or the HTTP transport path.

The evaluation fixture defines 16 scenarios covering mixed needs, urgent situations, food, health, employment, pension, education, fraud, family distress, low literacy, multilingual speech, and follow-up context. The rubric considers usefulness, grounding, understandable speech, one-question behaviour, handling of multiple needs, and privacy. Unsupported material claims and invented completed actions are hard failures. The existence of these fixtures does not mean all scenarios have passed [8].

The proposed pilot remains a future evaluation: recruit 8–12 participants across languages and dialects and compare two or three journeys with equivalent portal tasks. Completion, time, errors, and 1–5 clarity/trust feedback should be recorded. No completed participant study is claimed at this checkpoint.

3.2 Performance Metrics

3.2.1 Definition and Interpretation of Timing

The prototype uses turn-level Time-to-Guidance: the interval from the end of the citizen's request until grounded guidance is received, excluding connection and worker startup time:

$$\mathrm{TTG}_{\mathrm{turn}} = t_{\text{grounded answer received}} - t_{\text{citizen request finished}}.$$

The target median is **at most seven seconds** for standardised short requests requiring one lookup. The original proposal's connection-to-guidance metric includes startup and is a separate measure.

The backend's `connected_at` and `first_guidance_at` record an internal connection-to-tool proxy. Turn-level TTG instead requires aligned end-of-speech and grounded-answer delivery events; a tool result alone does not measure what the citizen heard.

Additional instrumentation separates:

- **Worker startup:** conversation creation and session startup, excluding initial room connection and HTTP client creation.
- **Retrieval time:** backend catalogue lookup and ranking duration.
- **Voice turn timing:** available SDK timing values, including time to first token, end-to-end turn latency, and playback latency.
- **Speech-end to speaking-state delay:** elapsed time between the detected end of citizen speech and the agent entering the speaking state; this excludes browser playout.

These clocks describe separate intervals and must not be combined into a single latency result.

3.2.2 Observed Results and Evaluation Targets

Table 3.1 combines locally checked implementation evidence with clearly attributed prior observations and proposed targets for the next evaluation stage. Target values are evaluation goals rather than measured results. The voice timings come from the repository runbook's account of one synthetic English call; they were not newly measured for this report [5].

Table 3.1: Observed prototype results and proposed evaluation targets.

Measure	**Value/status**	**Description**
Local automated tests	310 passed	Selected local suite, with two voice-runtime modules excluded; PostgreSQL-specific checks were outside the executed selection.
Lint	Passed	Ruff checked backend application, tests, scripts, and migrations.
Reference catalogue	18 human-verified records	Six categories; human verification confirmed by the project team. Stored review metadata awaits synchronisation.
Rule coverage	5 records	Rule sets exist in the reviewed catalogue; deployment publication is not established by this count.
Document-field coverage	18/18 records	All scheme records define a structured document field.
Voice scenarios	16 defined	Evaluation inputs and rubric exist; no complete scenario pass rate has been established.
Agent readiness	2,431 ms	Prior runbook observation from one successful synthetic English call; not a distribution or reliability target.
Speech-end to first returned audio	3,064 ms	Same prior call; not equivalent to grounded-answer delivery or browser playout timing.
Median turn-level TTG	Target: ≤ 7 seconds	Median time from the end of the citizen's request until grounded guidance reaches them. Connection and worker startup are excluded; evaluated on short requests requiring one lookup.
Citation correctness	Target: $\geq 95\%$	Percentage of evaluated factual claims supported by their cited official sources.
Handoff precision/recall	Targets: precision $\geq 90\%$; recall $\geq 90\%$	Precision measures appropriate handoffs among all created handoffs. Recall measures detected cases among all cases requiring human assistance.
Throughput and replica scaling	Targets: 20, 35, 55 requests/s	Completed API requests per second with 1, 2, and 4 replicas, respectively, under comparable mocked-model workloads. Record response latency and errors alongside throughput.
User clarity	Target: mean $\geq 4/5$	Participants rate how understandable the guidance and next steps are on a 1–5 scale during the planned pilot.

3.3 Analysis

3.3.1 Achievement of Prototype Objectives

The backend now supports a complete sequence of typed operations: conversation creation, scheme discovery, deterministic guidance over published records, handoff creation, operator updates, and audit inspection. The selected local test run provides evidence for these application contracts. The reference catalogue and ranking logic also enable useful retrieval during development. Human review of all 18 initial records is complete; review metadata synchronisation and database publication remain separate operational steps.

The voice worker and supporting scripts establish an integration path, and the runbook records a successful synthetic voice exchange. However, this remains partial evidence for the voice objective. The same documentation reports a subsequent integrated call that failed to complete guidance, and the voice-design notes record inconsistent behaviour in complex requests. Full phone accessibility, verified catalogue deployment, and representative speech reliability are therefore not completed objectives.

3.3.2 Recent Development Progress

The report incorporates the latest repository history rather than carrying forward the earlier proposal's future-tense claims as completed work:

- **Commit `595fc75`:** introduced the voice integration, browser harness, and deployment foundations.
- **Commit `992d638`:** added the Week 5 backend test suite and accompanying verification journal.
- **Commit `def3630`:** expanded the catalogue and retrieval, refined Gemini voice behaviour and timing, strengthened concurrency and numeric validation, added PostgreSQL and voice tests, and updated documentation.
- **Commit `e34544d`:** merged the test work and resolved the reported test merge conflict; this is the revision used for the report.

3.3.3 Challenges and Design Responses

Balancing helpfulness and verification. The initial verified-only path requires reviewed data before it can answer useful questions. All 18 initial records have now been human-verified. The prototype exposes local references with their stored review metadata while preserving the publication gate for verified tools. Synchronising the earlier pending metadata with completed review is necessary. This improves development-time usefulness but increases the need to test whether spoken answers preserve uncertainty and material conditions.

Understanding ordinary and mixed needs. Users may describe several problems without naming a scheme. Multilingual phrase expansion, curated need concepts, and combined ranking improve retrieval for such inputs. These mappings remain finite and require extension and evaluation for unseen phrasing, regional languages, and state-specific support.

Managing voice latency and lifecycle. The context response mode removes an intermediate text-answer generation step from the voice path. Timing events distinguish retrieval, startup, and turn delays. Asynchronous transcript writes and single-task cleanup improve lifecycle handling. These are implemented mechanisms; a statistically supported latency improvement still requires comparable before-and-after runs.

Preserving consistency under concurrency. Application state is persisted and critical transitions use database locks. PostgreSQL migration and concurrency tests now exist and are configured in CI. The local SQLite run cannot validate those PostgreSQL behaviours, so the report does not infer concurrency capacity from its pass count.

3.3.4 Limitations at the Prototype Checkpoint

1. The browser channel does not satisfy the ordinary-phone accessibility objective, and there is no live telephone transfer.
2. Document and rule coverage are incomplete, and the full source-import workflow is unfinished.
3. Real-model answer quality and multilingual behaviour remain uneven. Documentation reports wrong-language responses, missed retrieval, multiple questions, or omitted material conditions in some trials.
4. Integrated live sessions have shown intermittent 1011 errors. A successful direct SDK call does not establish the cause of an integrated failure or prove that the browser voice path is reliable.
5. Transcript privacy requires enforced controls beyond prompt instructions, and staging deployment and operational monitoring remain incomplete.

These limitations define the next engineering stage. They do not invalidate the tested backend mechanisms, but they constrain the claims that can be made about citizen readiness.

Chapter 4

Conclusion

4.1 Summary of Work

Janmitra has progressed from a proposed phone-accessible guidance system to an implemented browser voice prototype supported by a modular backend. The work includes typed guidance tools, a structured reference catalogue, deterministic eligibility and document logic, preserved publication history, operator handoff records, audit trails, migrations, containers, CI checks, and voice-evaluation support. The latest changes substantially expand retrieval and improve the separation between published records and local reference material. The project team confirms human verification of all 18 initial scheme records.

The selected local verification run passed 310 tests and lint checks. The report also identifies the unexecuted test scope and the limited nature of prior live voice observations. This makes the checkpoint a documented implementation result without treating future pilot or scalability targets as measured achievements.

4.2 Key Findings

- **Explicit tool boundaries make guidance testable.** Eligibility, publication, and queued handoff creation can be checked independently of speech generation and provider availability.
- **Data review is an operational dependency.** Schema-valid JSON and official-source links are useful foundations, but they do not replace a human check of the actual scheme conditions.
- **Retrieval must preserve the citizen's situation.** Combined ranking and need-based matching help retain multiple concerns, while uncertainty and record provenance must survive into the spoken response.
- **Different evidence answers different questions.** Unit tests establish application behaviour; live audio trials, semantic review, pilot feedback, and load testing establish separate aspects of usability and capacity.

4.3 Future Work and Recommendations

1. **Expand catalogue coverage.** Add more state-specific schemes, complete missing document checklists, and extend eligibility rule coverage.
2. **Complete the ingestion workflow.** Add official-source import, retained snapshots, AI-assisted extraction into drafts, reviewer comparison, and controlled publication with visible history.
3. **Validate voice quality and reliability.** Run the full 16-scenario set, assess transcripts and audio semantically, exercise follow-up context and interruptions, and investigate intermittent integrated-call failures.
4. **Resolve the channel scope.** Implement and validate phone/SIP access if the original ordinary-phone objective is retained. If browser access becomes the final scope, explicitly revise the accessibility claims and pilot assumptions rather than describing it as an equivalent phone service.
5. **Demonstrate scalability.** Build the mocked-model load harness, compare 1, 2, and 4 API replicas under equivalent conditions, identify the first bottleneck, apply a justified fix, and report before-and-after results. Measure voice-worker capacity separately from HTTP capacity.
6. **Prepare a controlled pilot deployment.** Add staging CD, operational checks, transcript redaction and retention controls, and a documented operator process before recruiting the proposed 8–12 users.

4.4 Final Remarks

The prototype demonstrates how conversational AI can be connected to structured public-service information and deterministic backend operations. Its main engineering contribution at this stage is the separation of speech interaction, retrieval, trusted decisions, and persistent records into components that can be examined and tested. Publishing the human-verified catalogue and evaluating real citizen journeys are the next steps toward the proposal's wider goal of accessible, dependable civic guidance.

Bibliography

[1] Dhruv Srivastava, Harkamal Singh Lubana, and Paras. *Janmitra: A Voice-First Civic Scheme Guidance Platform for Rural India*. UCS503P project proposal. Repository source: `project-proposal/main.tex`.

[2] Janmitra project team. *Prototype implementation and commit history*. Repository revision `e34544d`, with major updates in `def3630`. Application, tests, scripts, migrations, and data under `code/backend/`; CI configuration in `.github/workflows/backend.yml`.

[3] Janmitra project team. *Architecture*. `docs/architecture.md`, revision `e34544d`.

[4] Janmitra project team. *Browser Voice Architecture*. `docs/voice-architecture.md`, revision `e34544d`.

[5] Janmitra project team. *Run and Test Janmitra*. `docs/run-and-test.md`, including the documented Gemini 3.1 provider-status observations, revision `e34544d`.

[6] Janmitra project team. *Catalogue Review Checklist*. `code/backend/data/REVIEW.md` and accompanying scheme JSON records, revision `e34544d`.

[7] Janmitra repository. *Automated Source Check: 9 October 2026*. `code/backend/data/SOURCE-CHECK-2026-10-09.md`. Automated research notes supplied for subsequent human review.

[8] Janmitra project team. *Voice Evaluation Scenarios and Rubric*. `code/backend/scripts/fixtures/voice-evaluation.json`, with `code/backend/scripts/evaluate_voice_answer.py` and `code/backend/scripts/smoke_voice.py`, revision `e34544d`.